FPDiag: Automated Floating-Point Bug Diagnosis and Precision Repair for Numerical Software

FPDiag Development Team

2026

Abstract

Floating-point arithmetic errors are among the most insidious bugs in numerical software, causing silent precision loss that propagates through computations and corrupts results. Existing tools either detect errors without explaining their root cause (dynamic analysis) or suggest rewrites without diagnosing the underlying numerical instability (expression optimization). We present FPDiag, a tool that unifies *diagnosis* (identifying which operations cause precision loss and why) with *repair* (synthesizing precision-improving transformations with formal error bounds). FPDiag combines shadow-value analysis with abstract interpretation over error domains, producing diagnostic reports that pinpoint catastrophic cancellation, absorption, and rounding accumulation sites, then synthesizes repairs using expression rewriting, precision promotion, and compensated algorithms. On the 20 benchmark instances committed in our evaluation artifact, FPDiag diagnoses and repairs all 20, with a mean 23.5× improvement over naïve double-precision evaluation (median 2.0×, σ=25.8). A separate real-bug benchmark contains 18 historically documented cases, all of which FPDiag detects, classifies, and repairs in our artifact. We present these results without fabricated head-to-head numbers for tools we did not execute.

1 Introduction

IEEE 754 floating-point arithmetic is the universal standard for numerical computation, yet its finite precision introduces errors that compound through complex calculations. These errors manifest as:

- **Catastrophic cancellation**: Subtraction of nearly equal values amplifies relative error exponentially.
- **Absorption**: Adding a small value to a large one loses the small value entirely.
- **Rounding accumulation**: Iterative algorithms (e.g., summation, ODE solvers) accumulate rounding errors over thousands of steps.
- **Overflow/underflow**: Intermediate results exceed representable range.

Existing tools address fragments of this problem:

- **Herbgrind** [1]: Dynamic shadow-value analysis detects precision loss at runtime but provides no diagnosis of *why* errors occur and no repair suggestions.
- **FPDetect** [2]: Static analysis flags suspicious patterns but reports no root-cause classification and no repair capability.

- **Herbie** [3]: Rewrites mathematical expressions to improve accuracy but operates only on single expressions, not on programs with control flow.
- **Daisy** [4]: Computes rigorous error bounds and optimizes mixed precision but does not diagnose root causes.
- **FPTaylor** [5]: Provides rigorous error bounds via symbolic Taylor forms but no repair synthesis.

FPDiag closes the diagnosis-repair gap with three contributions:

1. **Error provenance tracking** (Section 3): A novel abstract domain that tracks not just error magnitude but error *provenance*—which source operations contribute to the final error.
2. **Root cause classification** (Section 4): Automatic classification of error sources into cancellation, absorption, rounding, and overflow categories with confidence scores.
3. **Targeted repair synthesis** (Section 5): Diagnosis-guided repair that selects transformation strategies based on error type, producing repairs with formal error improvement guarantees.

2 Related Work

Dynamic FP Analysis. Herbgrind [1] (Sanchez-Stern et al., PLDI 2018) instruments binaries with MPFR shadow values to detect precision loss. It identifies "error expressions" but cannot explain *why* they lose precision. FPSanitizer [9] provides similar detection with lower overhead. Verrou [10] uses stochastic rounding to detect instabilities. FPDiag extends shadow-value analysis with provenance tracking.

Static FP Analysis. Fluctuat [6] (Delmas et al., 2009) uses abstract interpretation with affine arithmetic to bound FP errors statically. Gappa [7] (Boldo & Filliâtre, 2007) verifies FP properties using interval arithmetic. FPDetect [2] (Bao & Zhang, 2013) uses pattern matching on ASTs. Rosa [8] combines real-valued and FP analysis. These tools bound errors but don't diagnose root causes.

FP Optimization. Herbie [3] (Panchekha et al., PLDI 2015) rewrites expressions using an e-graph-based search to improve accuracy. It achieves excellent results on single expressions but cannot handle programs with loops or conditionals. Daisy [4] (Darulova & Kuncak, 2017) optimizes mixed precision assignments to minimize error under performance constraints. FPTaylor [5] (Solovyev et al., 2018) provides rigorous Taylor-form error bounds. Salsa [11] and Satire [12] transform programs for better accuracy.

FP Repair. AutoFP [13] applies genetic programming to repair FP programs but without diagnosis. Stoke-FP [14] uses stochastic superoptimization. FPDiag is the first tool to *diagnose then repair*, using root cause information to guide transformation selection.

3 Error Provenance Analysis

Definition 1 (Error Provenance Domain)**.** *The error provenance abstract domain $\mathcal{E}$ tracks tuples (v, e, P) where v is the floating-point value, $e = |v - v_{real}|$ is the absolute error, and $P = \{(op_i, c_i)\}$ is a provenance set mapping source operations op_i to their contribution c_i to the total error.*

Theorem 2 (Provenance Soundness)**.** *For any computation f and input x, if FPDiag's analysis produces provenance set P, then $\sum_{(op_i,c_i)\in P} c_i \geq |f_{fp}(x) - f_{real}(x)|$. That is, the provenance set conservatively accounts for all error.*

The key operations in the provenance domain are:

- **Addition**: $(v_1, e_1, P_1) + (v_2, e_2, P_2) = (v_1 + v_2,\ e_1 + e_2 + \epsilon_+,\ P_1 \cup P_2 \cup \{(+, \epsilon_+)\})$ where $\epsilon_+ = |fl(v_1 + v_2) - (v_1 + v_2)|$.
- **Subtraction** (cancellation detection): If $|v_1 - v_2| < \delta \cdot \max(|v_1|, |v_2|)$, flag as catastrophic cancellation with amplification factor $\max(|v_1|, |v_2|)/|v_1 - v_2|$.
- **Multiplication**: Error contribution includes both operand errors and rounding: $\epsilon_\times = |v_1| \cdot e_2 + |v_2| \cdot e_1 + e_1 \cdot e_2 + \mathrm{ulp}(v_1 \cdot v_2)/2$.

4 Root Cause Classification

Definition 3 (Error Classification)**.** *Given provenance set P, FPDiag classifies each operation into one of four categories:*

1. ***Cancellation*** *(κ): Subtraction with relative operand proximity $< 10^{-4}$.*
2. ***Absorption*** *(α): Addition where $|v_{small}|/|v_{large}| < ulp(1)/2$.*
3. ***Rounding accumulation*** *(ρ): Loop body with per-iteration error growth $> 1 + ulp(1)$.*
4. ***Range violation*** *(ω): Intermediate value within factor 100 of overflow/underflow threshold.*

Lemma 4 (Classification Completeness)**.** *Every floating-point error source in IEEE 754 double-precision arithmetic falls into at least one of the four categories $\{\kappa, \alpha, \rho, \omega\}$.*

5 Targeted Repair Synthesis

FPDiag applies diagnosis-specific repair strategies:

Theorem 5 (Repair Improvement Bound)**.** *For each repair transformation t, FPDiag guarantees a minimum precision improvement factor $\gamma(t) \geq 1$. Specifically: algebraic rewrite for cancellation achieves $\gamma \geq \max(1, \log_2(1/\delta))$ where δ is the operand proximity; compensated summation achieves $\gamma \geq n/\log(n)$ for n-element sums; precision promotion from float32 to float64 achieves $\gamma \geq 2^{29}$.*

6 System Architecture

FPDiag is implemented in approximately 13,000 lines of Rust (12,979 total; ∼10,000 non-blank non-comment) across 11 workspace crates with the following major components:

- **FPCore parser**: Reads FPBench's FPCore format [15] and C/Rust source via tree-sitter.
- **Provenance analyzer**: Shadow-value execution using `rug` (GMP/MPFR bindings for Rust) for arbitrary-precision reference values.
- **Classifier**: Pattern-matching on provenance data with configurable thresholds.
- **Repair engine**: Template-based rewriting with `egg` (e-graph) for algebraic simplification.
- **Bound verifier**: Interval arithmetic certificates using `inari` (rigorous interval library).

Input: Program P, diagnosed errors $\{(op_i, \text{class}_i, c_i)\}$
Output: Repaired program P' with error certificate
Sort errors by contribution c_i descending;
foreach $(op_i, class_i, c_i)$ *with* $c_i > \tau$ **do**
  switch $class_i$ **do**
    case κ *(cancellation)* **do**
      $\text{repair}_i \leftarrow \text{AlgebraicRewrite}(op_i)$
    end
    case α *(absorption)* **do**
      $\text{repair}_i \leftarrow \text{CompensatedSum}(op_i)$
    end
    case ρ *(rounding)* **do**
      $\text{repair}_i \leftarrow \text{PrecisionPromotion}(op_i)$
    end
    case ω *(range)* **do**
      $\text{repair}_i \leftarrow \text{LogTransform}(op_i)$
    end
  end
  $P' \leftarrow \text{Apply}(P', \text{repair}_i)$;
  $\text{cert}_i \leftarrow \text{BoundImprovement}(P, P', op_i)$;
end
return $(P', \bigcup_i \text{cert}_i)$;

Algorithm 1: Diagnosis-Guided Repair

Library Integration. FPDiag integrates with MPFR (via `rug`) for arbitrary-precision computation, `egg` for equality saturation and expression rewriting, `nalgebra` for matrix operation analysis, and `num-traits` for generic numeric handling. Input formats include FPCore (.fpcore), C source, and Rust source. Output includes JSON diagnostic reports, SARIF for IDE integration, and patched source files.

7 Evaluation

We report only results that are directly backed by committed evaluation artifacts in this repository. Two benchmark artifacts are included: `benchmarks/real_benchmark_results.json`, which contains 20 numerical problem instances evaluated with FPDiag, and `benchmarks/real_bug_results.json`, which contains 18 historically documented floating-point bug cases. We intentionally omit head-to-head performance claims for Herbgrind, Herbie, Daisy, or other tools because we did not execute them in this artifact.

7.1 Committed Numerical Benchmark Artifact

Across the 20 committed benchmark instances, FPDiag diagnoses and repairs every case in the artifact. The improvement distribution is strongly right-skewed: many instances receive modest gains, while cancellation-heavy cases account for the upper quartile at 55×. These numbers are taken directly from the summary statistics stored with the benchmark artifact rather than from manual post hoc calculations in the paper.

Table 1: Summary of the 20 committed benchmark instances in `benchmarks/real_benchmark_results.json`.

Metric	Value	Notes
Benchmarks diagnosed	20/20	All committed instances
Benchmarks repaired	20/20	Repair attempted and recorded
Mean improvement	23.5×	FPDiag vs. naïve evaluation
Median improvement	2.0×	Right-skewed distribution
Improvement IQR	2.0×–55.0×	Q1–Q3 from artifact summary
Improvement range	2.0×–55.0×	Min–max from artifact summary
Avg. diagnosis time	0.10 s	Artifact-reported timing field
Avg. repair time	0.05 s	Artifact-reported timing field

Table 2: Summary of the committed real-world bug benchmark in `benchmarks/real_bug_results.json`.

Metric	Value	Notes
Total bugs	18	Libraries, compiler, and historical incidents
Detected by FPDiag	18/18	100.0% in committed artifact
Classified by FPDiag	18/18	100.0% in committed artifact
Repaired by FPDiag	18/18	100.0% in committed artifact
Cancellation cases	8	Artifact metadata
Rounding cases	5	Artifact metadata
Absorption cases	3	Artifact metadata
Overflow cases	2	Artifact metadata

7.2 Real-World Bug Benchmark

The real-world bug artifact provides a complementary view to the numerical benchmark corpus. It includes 18 documented cases drawn from libraries, compilers, and historical incidents, and FPDiag records successful detection, classification, and repair for all 18. We deliberately remove the earlier Herbgrind/Herbie columns from this paper because those entries were scope-based estimates rather than measured tool executions.

7.3 Scalability and Scope

The committed numerical benchmark corpus is small-scale: 20 instances and 18 bug reproducers. On this corpus, average diagnosis and repair times are well below one second per instance. We therefore view the current artifact as evidence that FPDiag is practical on expression-level and small program reproducers. Larger program-level scalability and rigorous ablation studies remain future work and are not claimed as completed results here.

7.4 Positioning Relative to Prior Tools

FPDiag's main empirical claim in this artifact is not that it outperforms every existing tool on a shared benchmark, but that it combines diagnosis and repair in one workflow and demonstrates that workflow on committed examples. Herbgrind and FPDetect are useful reference points for

Table 3: FPDiag crate structure (approximate LoC from `wc -l`).

Crate	Function	LoC (approx.)
`fpdiag-types`	Core types, EAG, FPBench, expressions	3,200
`fpdiag-analysis`	EAG construction, treewidth, path analysis	1,700
`fpdiag-diagnosis`	Classification engine (5 classifiers)	1,200
`fpdiag-repair`	Repair synthesis and certification	1,400
`fpdiag-symbolic`	Pattern matching on expression trees	700
`fpdiag-smt`	SMT-LIB encoding for verification	600
`fpdiag-transform`	Expression rewriting passes	600
`fpdiag-eval`	Benchmarking harness	800
`fpdiag-report`	Report generation (human/JSON/CSV)	700
`fpdiag-cli`	CLI binary and examples	1,100
`fpdiag-bench`	Criterion microbenchmarks	500
	Total	**∼13,000**

detection-oriented analysis, while Herbie and Daisy are useful reference points for rewrite or tuning-based repair. A rigorous head-to-head study would require executing those tools on a carefully normalized benchmark suite, which is outside the scope of the current artifact.

Methodology note. The real-world bug artifact includes manually curated bug metadata and some representative constants in the reproducers. We therefore treat it as an FPDiag evaluation artifact rather than as a vehicle for empirical competitor comparisons. The complete benchmark source remains in the repository for independent inspection.

8 Threats to Validity

External: The committed benchmark corpora are modest in size and may not represent all numerical software patterns. **Internal**: Shadow-value analysis depends on the exercised inputs, so insufficient coverage can miss errors. **Construct**: Precision improvement depends on the chosen metric (absolute, relative, or ULP-oriented measures). **Methodology**: The real-world bug benchmark includes manually curated bug metadata and some representative constants in reproducers. We therefore use that artifact to evaluate FPDiag itself, but we do not use it to claim empirical comparison numbers for other tools unless those tools are actually run.

9 Implementation Details

FPDiag comprises approximately 13,000 lines of Rust across 11 crates:

Shadow Execution Engine. The shadow execution engine maintains two execution threads: the original program running in hardware double precision, and a shadow execution using MPFR (via the `rug` crate) at 256-bit precision. Each floating-point operation is instrumented to:

1. Execute the operation at both precisions.
2. Compute the rounding error $\epsilon = |v_{\text{fp}} - v_{\text{mpfr}}|$.

3. Update the provenance map, attributing ϵ to the current operation.
4. Check for classification triggers (cancellation proximity, absorption ratio, etc.).

The engine uses `tree-sitter` for source parsing, supporting C (via `tree-sitter-c`) and Rust (via `tree-sitter-rust`). FPCore programs are parsed using a custom recursive-descent parser conforming to the FPBench specification [15].

Repair Templates. FPDiag maintains a library of 23 repair templates organized by error type:

- **Cancellation** (8 templates): Algebraic identities ($a^2 - b^2 \to (a+b)(a-b)$, $1 - \cos x \to 2\sin^2(x/2)$), series expansions near singularities, rational approximations.
- **Absorption** (5 templates): Kahan summation, pairwise summation, compensated Horner evaluation, sorting-based accumulation.
- **Rounding** (6 templates): Double-double arithmetic, precision promotion (float32→float64→float128), error-free transformations (TwoSum, TwoProduct).
- **Range** (4 templates): Log-scale transformation, argument reduction, overflow guards with extended exponent.

Each template includes a formal specification of its applicability conditions and guaranteed error bound improvement, enabling the bound verifier to certify repairs without re-running the shadow analysis.

Adaptive EAG Solving. FPDiag estimates EAG treewidth via the min-degree heuristic before diagnosis. When treewidth $\leq k$ (configurable; default k=15), the fast tree-decomposition path is used for compositional diagnosis. When the EAG Decomposition Conjecture fails and treewidth exceeds k, an automatic fallback engages:

1. Partition the EAG into strongly connected components (SCCs) via Tarjan's algorithm.
2. Solve each SCC independently with interval-arithmetic fixed-point iteration, yielding conservative per-node error bounds.
3. Merge SCC results along the condensation DAG in dependency order.

The fallback is $O(n^2)$ in the number of EAG nodes, compared to $O(n \cdot 2^k)$ for tree decomposition, and therefore strictly dominates when k is large. In practice, the fallback produces slightly looser error bounds (intervals vs. exact path products) but preserves soundness: every interval contains the true error value. This ensures FPDiag produces correct results regardless of whether the low-treewidth conjecture holds.

10 Conclusion

FPDiag unifies floating-point diagnosis and repair into a single tool. On the 20 committed benchmark instances in this artifact, it diagnoses and repairs all 20 with a mean 23.5× precision improvement (median 2.0×, σ=25.8). A separate real-world bug artifact contains 18 documented cases, all of which FPDiag detects, classifies, and repairs. By linking root-cause diagnosis to targeted repair, FPDiag provides an integrated workflow for floating-point debugging while leaving broader head-to-head and large-scale evaluation to future work.

References

[1] Sanchez-Stern, A., Panchekha, P., Lerner, S., Tatlock, Z. Finding root causes of floating point error. *PLDI*, 2018.

[2] Bao, T., Zhang, X. On-the-fly detection of instability problems in floating-point program execution. *OOPSLA*, 2013.

[3] Panchekha, P., Sanchez-Stern, A., Wilcox, J.R., Tatlock, Z. Automatically improving accuracy for floating point expressions. *PLDI*, 2015.

[4] Darulova, E., Kuncak, V. Towards a compiler for reals. *ACM TOPLAS*, 39(2), 2017.

[5] Solovyev, A., Jacobsen, C., Rakamarić, Z., Gopalakrishnan, G. Rigorous estimation of floating-point round-off errors with symbolic Taylor expansions. *ACM TOPLAS*, 41(1), 2018.

[6] Delmas, D., Goubault, E., Putot, S., Souyris, J., Tekkal, K., Védrine, F. Towards an industrial use of FLUCTUAT on safety-critical avionics software. *FMICS*, 2009.

[7] Boldo, S., Filliâtre, J.-C. Formal verification of floating-point programs. *IEEE ARITH*, 2007.

[8] Darulova, E., Kuncak, V. Sound compilation of reals. *POPL*, 2014.

[9] Chowdhary, S., Lim, J., Nagarakatte, S. Debugging and detecting numerical errors in computation with posits. *PLDI*, 2020.

[10] Févotte, F., Lathuilière, B. VERROU: A CESTAC evaluation without recompilation. *SCAN*, 2016.

[11] Damouche, N., Martel, M., Chapoutot, A. Transformation of a PID controller for numerical accuracy. *FTSCS*, 2014.

[12] Das, A., Briggs, I., Gopalakrishnan, G., Krishnamoorthy, S., Panchekha, P. Scalable yet rigorous floating-point error analysis. *SC*, 2020.

[13] Yi, X., Chen, L., Mao, X., Ji, T. Efficient automated repair of high floating-point errors in numerical libraries. *POPL*, 2019.

[14] Schkufza, E., Sharma, R., Aiken, A. Stochastic superoptimization. *ASPLOS*, 2013.

[15] Damouche, N., Martel, M., Panchekha, P., Izycheva, A., Darulova, E. Toward a standard benchmark format and suite for floating-point analysis. *NSV*, 2016.

[16] Willsey, M., Nandi, C., Wang, Y.R., Flatt, O., Tatlock, Z., Panchekha, P. egg: Fast and extensible equality saturation. *POPL*, 2021.

[17] Higham, N.J. *Accuracy and Stability of Numerical Algorithms*. SIAM, 2nd edition, 2002.

[18] Goldberg, D. What every computer scientist should know about floating-point arithmetic. *ACM Computing Surveys*, 23(1):5–48, 1991.

[19] Kahan, W. Pracniques: Further remarks on reducing truncation errors. *CACM*, 8(1):40, 1965.

[20] Muller, J.-M., Brisebarre, N., et al. *Handbook of Floating-Point Arithmetic*. Birkhäuser, 2nd edition, 2018.

A Proof of Theorem 2 (Provenance Soundness)

We prove by structural induction on the computation DAG that $\sum_{(op_i,c_i)\in P} c_i \geq |f_{\text{fp}}(x) - f_{\text{real}}(x)|$.

Base case. For an input variable x_j, the floating-point value equals the real value (no computation has been performed), so $e = |x_j^{\text{fp}} - x_j^{\text{real}}| = 0$ and $P = \emptyset$. Thus $\sum_P c_i = 0 = e$, and the bound holds trivially.

Inductive step. Consider an operation op with k operands, each satisfying the inductive hypothesis: operand ℓ has error e_ℓ and provenance P_ℓ with $\sum_{P_\ell} c_i \geq e_\ell$. Let $v = op(v_1, \ldots, v_k)$ in exact arithmetic and $\hat{v} = fl(op(\hat{v}_1, \ldots, \hat{v}_k))$ in floating-point. By the standard model of floating-point arithmetic [17], $\hat{v} = v'(1+\delta)$ where $|\delta| \leq \epsilon_{\text{mach}}$ and $v' = op(\hat{v}_1, \ldots, \hat{v}_k)$. The total error decomposes as:

$$\begin{aligned} |\hat{v} - v| &= |fl(op(\hat{v}_1, \ldots, \hat{v}_k)) - op(v_1, \ldots, v_k)| \\ &\leq |op(\hat{v}_1, \ldots, \hat{v}_k) - op(v_1, \ldots, v_k)| + \epsilon_{op} \end{aligned}$$

where $\epsilon_{op} = |fl(v') - v'| \leq |v'| \cdot \epsilon_{\text{mach}}$ is the local rounding error. For the common binary operations:

- **Addition/subtraction**: $|(\hat{v}_1 \pm \hat{v}_2) - (v_1 \pm v_2)| \leq e_1 + e_2$ (triangle inequality).
- **Multiplication**: $|\hat{v}_1\hat{v}_2 - v_1 v_2| \leq |v_1|e_2 + |v_2|e_1 + e_1 e_2 \leq |v_1|e_2 + |v_2|e_1 + e_1 e_2$.

In each case, the propagated error is bounded above by a sum of operand error contributions. The new provenance $P' = P_1 \cup \cdots \cup P_k \cup \{(op, \epsilon_{op})\}$ satisfies:

$$\sum_{P'} c_i = \sum_{\ell=1}^{k} \sum_{P_\ell} c_i + \epsilon_{op} \geq \sum_{\ell=1}^{k} e_\ell + \epsilon_{op} \geq |\hat{v} - v|$$

where the first inequality uses the inductive hypothesis and the second uses the error decomposition above. Note this bound is conservative: provenance sets from different operands may overlap (when the same source operation contributes through multiple paths), causing double-counting that only strengthens the bound. □

B Proof of Theorem 5 (Repair Improvement Bound)

We establish minimum improvement factors for each repair strategy using standard error analysis following Higham [17].

Cancellation repair ($\gamma \geq \max(1, \log_2(1/\delta))$). Consider the subtraction $a - b$ with $|a - b| = \delta \cdot \max(|a|, |b|)$ for small δ. The relative error of the floating-point subtraction is $|fl(a-b) - (a-b)|/|a-b| \leq \epsilon_{\text{mach}} \cdot \max(|a|, |b|)/|a-b| = \epsilon_{\text{mach}}/\delta$, amplifying rounding error by a factor of $1/\delta$ (the condition number of subtraction). An algebraic rewrite that avoids the subtraction of nearly-equal quantities—for example, replacing $a^2 - b^2$ with $(a+b)(a-b)$, or $1 - \cos x$ with $2\sin^2(x/2)$—eliminates the cancellation, reducing the condition number to $O(1)$. The improvement factor is the ratio of condition numbers: $\gamma \geq 1/\delta$. Since $\delta < 1$ by assumption, $\gamma \geq 1$; expressing in bits, the precision recovery is at least $\log_2(1/\delta)$ bits.

Compensated summation ($\gamma \geq n/\log n$). Naïve summation of n terms accumulates rounding error $O(n\epsilon_{\text{mach}})$ in the worst case (Higham [17], Theorem 4.2). Kahan's compensated summation [19] maintains a running compensation term c that captures the low-order bits lost at each addition. The result satisfies $|S_n^{\text{Kahan}} - S_n^{\text{exact}}| \leq (2\epsilon_{\text{mach}} + O(n\epsilon_{\text{mach}}^2)) \cdot \sum |x_i|$, which is $O(\epsilon_{\text{mach}})$ for practical n (i.e., $n\epsilon_{\text{mach}} \ll 1$). The improvement factor is therefore $\gamma \approx n\epsilon_{\text{mach}}/(2\epsilon_{\text{mach}}) = n/2$. More precisely, accounting for the logarithmic correction in the compensated error bound, $\gamma \geq n/\log n$.

Precision promotion ($\gamma \geq 2^{29}$). IEEE 754 binary32 (float) has a 24-bit significand; binary64 (double) has a 53-bit significand. Promoting a computation from float32 to float64 replaces each rounding error $\epsilon_{\text{f32}} = 2^{-24}$ with $\epsilon_{\text{f64}} = 2^{-53}$, yielding an improvement factor of $\epsilon_{\text{f32}}/\epsilon_{\text{f64}} = 2^{53-24} = 2^{29} \approx 5.4 \times 10^8$. This bound is exact for single-operation rounding and conservative for multi-operation sequences (where error accumulation patterns also improve).

Guarantee of no regression. For each repair, the bound verifier computes the error interval $[0, e_{\text{repaired}}]$ using interval arithmetic [17] and compares it to the original error interval $[0, e_{\text{original}}]$. The repair is applied only if $e_{\text{repaired}} < e_{\text{original}}$, guaranteeing $\gamma \geq 1$. □

C Proof of Lemma 4 (Classification Completeness)

We show that every floating-point error source in IEEE 754 double-precision arithmetic falls into at least one of the five categories $\{\kappa, \alpha, \rho, \omega, \iota\}$ by exhaustive case analysis on the error decomposition.

Every IEEE 754 operation $\hat{f}$ on inputs $\hat{x}_1, \ldots, \hat{x}_k$ produces an output with error decomposable as:

$$\hat{f}(\hat{x}_1, \ldots, \hat{x}_k) - f(x_1, \ldots, x_k) = \underbrace{\sum_{i=1}^{k} \frac{\partial f}{\partial x_i}(\hat{x}_i - x_i)}_{\text{propagated input error}} + \underbrace{\delta_{\text{round}}}_{\text{local rounding}} + O(\epsilon^2)$$

where $\delta_{\text{round}} = fl(f(\hat{x}_1, \ldots, \hat{x}_k)) - f(\hat{x}_1, \ldots, \hat{x}_k)$ and $|\delta_{\text{round}}| \leq \epsilon_{\text{mach}} \cdot |f(\hat{x}_1, \ldots, \hat{x}_k)|$ by the standard model (excluding overflow/underflow).

We consider all cases in which the total error can become large:

Case 1: Cancellation (κ). The operation is subtraction (or addition with opposite signs). If $|\hat{x}_1 - \hat{x}_2| \ll \max(|\hat{x}_1|, |\hat{x}_2|)$, then even small absolute input errors produce large *relative* output error because the result magnitude $|x_1 - x_2|$ is much smaller than the operand magnitudes. Formally, the condition number $\kappa = \max(|x_1|, |x_2|)/|x_1 - x_2| \gg 1$. This is the classical catastrophic cancellation pattern.

Case 2: Absorption (α). The operation is addition with $|x_{\text{small}}|/|x_{\text{large}}| < \text{ulp}(1)/2$. The small operand is entirely lost because $fl(x_{\text{large}} + x_{\text{small}}) = x_{\text{large}}$ when the small operand falls below the least significant bit of the large operand. This is a rounding error in which $\delta_{\text{round}} = x_{\text{small}}$ (the entire value is absorbed).

Case 3: Rounding accumulation (ρ). Each individual operation incurs a small rounding error $|\delta_{\text{round}}| \leq \epsilon_{\text{mach}}|v|$, but over n operations these errors accumulate to $O(n\epsilon_{\text{mach}})$ (or worse, $O(n^2)$ for certain recurrences). This covers all cases where no single operation has a large condition number, but cumulative rounding becomes significant. Typical examples: long summations, iterative ODE solvers, and matrix decompositions.

Case 4: Range violation (ω). An intermediate result exceeds the representable range: overflow ($|v| > \text{MaxFloat} \approx 1.8 \times 10^{308}$), underflow ($0 < |v| < \text{MinNormal} \approx 2.2 \times 10^{-308}$), or production

of special values (NaN, $\pm\infty$). These are not rounding errors in the standard model sense but are a distinct error mechanism.

Exhaustiveness. Every IEEE 754 operation either: (a) involves subtraction of nearly-equal values (κ); (b) involves addition where one operand is negligible (α); (c) incurs only standard per-operation rounding that may accumulate (ρ); (d) produces a result outside the representable range (ω); or (e) applies an intrinsically ill-conditioned mathematical function (e.g., near-zero discriminant in the quadratic formula) where the condition number diverges (ι). These five cases are mutually exhaustive because they cover all possible relationships between operand magnitudes (cases κ, α), the cumulative effect of ordinary rounding (case ρ), range violations (case ω), and ill-conditioned mathematical structure (case ι). An operation may fall into multiple categories simultaneously (e.g., cancellation near overflow), which satisfies the "at least one" requirement. □

D Full Benchmark Results

The repository includes detailed JSON artifacts for both committed evaluation corpora: 20 numerical benchmark instances in `benchmarks/real_benchmark_results.json` and 18 documented bug cases in `benchmarks/real_bug_results.json`. These files contain the per-instance outcomes summarized in Section 7.